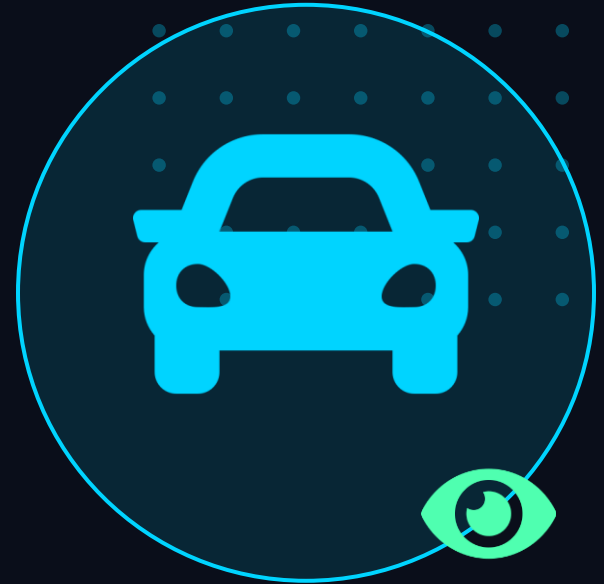


CAPSTONE PROJECT 2026

DRIVER MONITORING SYSTEM

AI-Powered Real-Time Driver Safety
using Computer Vision & Edge AI on Jetson Nano



Smart Car AI

- Farhan Hikmatullah Daulay 611451002
- Professor Cho Li

| Kishore Sridhar 611451003
| Professor Hsieh

10 June 2026



PROBLEM STATEMENT

Why does this system need to exist?

Cause of road fatalities worldwide

Visual Distraction & Improper Surveillance

Accidents from driver fatigue

Prevention from traditional safety



Drowsiness

Prolonged eye closure during long trips goes undetected



Distraction

Head turning beyond 20° left or right detected as loss of attention



Distraction

Phone use & loss of focus cause delayed reactions



Reactive Safety

Seatbelts & airbags act AFTER the crash — not before



MOTIVATION

What is the Solution? Who will get the benefit? Why edge AI?

So the solution is...

Safety-critical applications require immediate decision-making, making Edge AI the most practical and reliable deployment solution.

Benefit

- Private vehicle drivers
- Taxi and ride-hailing drivers
- Logistics and fleet operators
- Passengers and road users

PC Approach

- Requires internet connectivity
- Higher communication latency
- Privacy concerns from video transmission
- Bulky and unsuitable for vehicles
- Higher power consumption

VS

Edge AI Approach

- Fully offline operation
- Real-time response
- Driver data remains local
- Compact embedded deployment
- Low power operation (7–25W)

HARDWARE ARCHITECTURE

How the physical components connect and interact



Jetson Orin Nano Super

128-core Maxwell GPU, TensorRT



USB Camera

640×480 BGR async video feed



USB Speaker

1000Hz ALSA audio alarm output



Jumper Wires & Breadboard

Circuit connections



Vibration Motor

Haptic alert via GPIO Pin 33



LED Green, Yellow Red Module

Visual normal, warning, & critical alarm
Pin 13: Green, Pin 29: Yellow, Pin 31: Red



DIODE 1N4007

High-current safety control



10kΩ Resistors

GPIO Vibration Motor pin protection



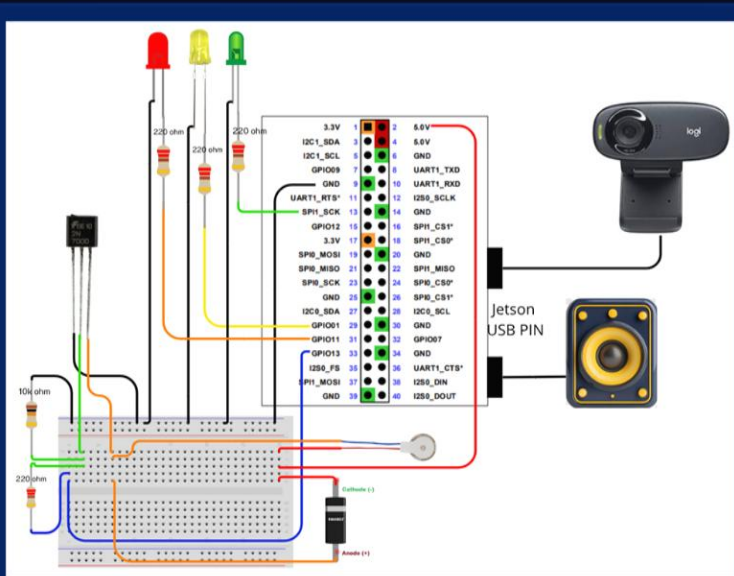
220Ω Resistors ×4

GPIO LED pin protection



MOSFET 2N7000

High-current safety control



SYSTEM ARCHITECTURE

5-Phase pipeline from sensing to action



WHAT CHANGED VS THE PROPOSAL

We proposed this and shipped this — here is why

Proposed	Shipped	Why?
YOLOv11n for driver behavior detection	Upgraded to YOLOv26n for improved detection performance	Model Upgrade: consider YOLOv26n during development and provided a more recent architecture with better performance than YOLOv11n.
IMX219 CSI Camera connected directly to Jetson	USB Camera deployment	Hardware Compatibility: The IMX219 camera could not be reliably connected and recognized by the Jetson Nano, so a USB camera was adopted to ensure stable operation.
Fixed EAR threshold for drowsiness detection	Dynamic eye calibration system using a personalized baseline	Improved Accuracy: Testing showed that a single fixed EAR threshold was not suitable for all users because eye sizes and facial characteristics vary between individuals.
MPU6050 for vehicle motion detection	Removed	Difficult to obtain consistent vehicle-motion measurements during testing and demonstrations.



AI MODELS — IMPLEMENTED

What models that used?

Selected Model	Alternative Model	Why
PFLD	MediaPipe Face Mesh	Lightweight, fast inference, suitable for real-time edge deployment
6DRepNet		Higher head-pose accuracy and direct yaw/pitch/roll estimation
Yolov26n	SSD-MobileNet	Better detection performance while maintaining lightweight deployment on Jetson

6DRepNet

6D Rotation Representation Network

- ↕ Pitch angle → Nodding off
- ↔ Yaw angle → Looking away
- 🎯 3D head pose in real-time

PFLD

Practical Facial Landmark Detection

- 👁️ Eye Aspect Ratio (EAR) → Drowsiness
- 👄 Mouth Aspect Ratio (MAR) → Yawning
- 📍 106 Facial Landmark Points

Yolov26n

Mobile Phone Detection

- 📱 Object Detection → Phone

PERFORMANCE TARGETS & BASELINE

Targets set upfront — then each optimization step and its measured impact

Project Targets

Metric	Target
Latency	< 500 ms
FPS	> 10 FPS
Power Consumption	< 25W
CPU Usage	< 80%
GPU Usage	< 100%

Edge Deployment Constraints:

- Real-time driver monitoring
- Embedded hardware limitations
- Low power operation
- Reliable safety response

Baseline System & Results



Metric	Baseline
Latency	100.2 ms
FPS	4 – 7 FPS
Power Consumption	< 8W
CPU Usage	< 77%
GPU Usage	< 98%

Baseline Configuration

- PyTorch inference
- Full frame processing
- Single-thread execution
- No TensorRT



OPTIMIZATION — IMPLEMENTED

TensorRT FP16 engines deployed on Jetson Nano Maxwell GPU

Camera

Multi-Threaded Processing

- Separate camera capture and inference tasks
- Parallel execution of system components
- Better CPU and GPU resource utilization

Frame Skipping

- Process selected frames instead of every frame
- Reduce redundant computations
- Improve throughput while maintaining detection reliability

Asynchronous Inference

- Camera continues capturing while AI models run
- Non-blocking processing pipeline
- Lower end-to-end latency

TensorRT FP16

- Converted models into TensorRT engines
- FP16 precision deployment
- Faster inference with minimal accuracy loss

Real Time Alert

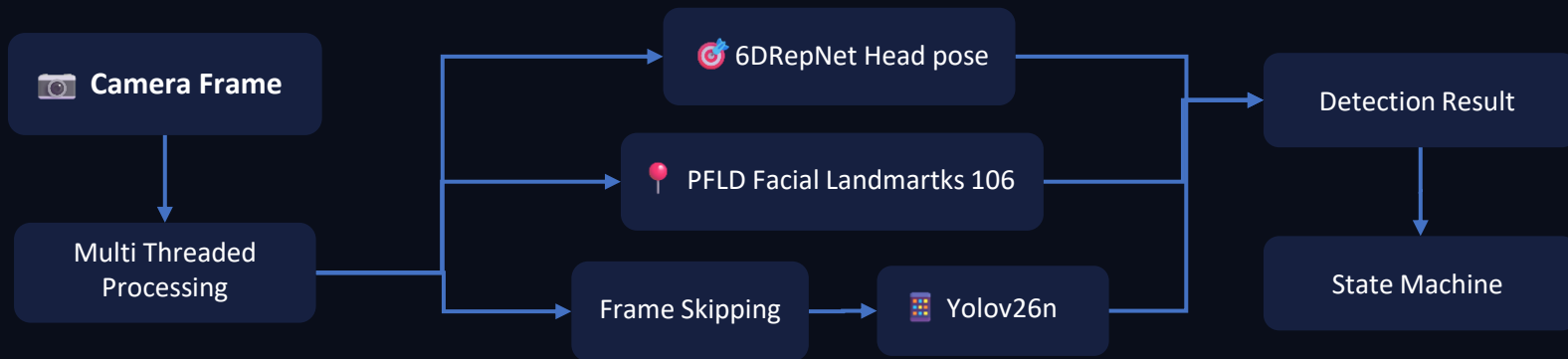
Lower Latency • Higher FPS • Efficient GPU Utilization

IMPLEMENTATION

How we built it — highlights, CI/CD, and test methodology

IMPLEMENTATION HIGHLIGHTS

6 things we built and are proud of



Challenge

Running multiple AI models simultaneously on Jetson Orin Nano can create inference bottlenecks and increase latency.

Solution

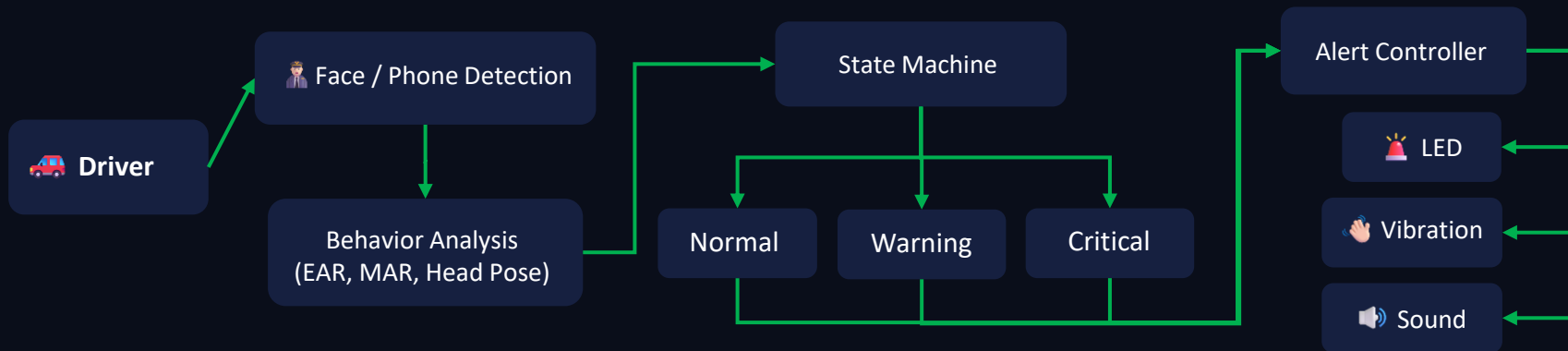
- Multi-threaded processing
- Asynchronous inference
- Frame skipping
- TensorRT FP16 acceleration

Result

- ✓ Continuous camera acquisition
- ✓ Non-blocking inference
- ✓ Reduced latency
- ✓ Real-time driver monitoring

IMPLEMENTATION HIGHLIGHTS

6 things we built and are proud of



Challenge

1. Fixed EAR thresholds are not reliable for all users
2. Detection alone cannot improve safety unless drivers receive immediate feedback

Solution

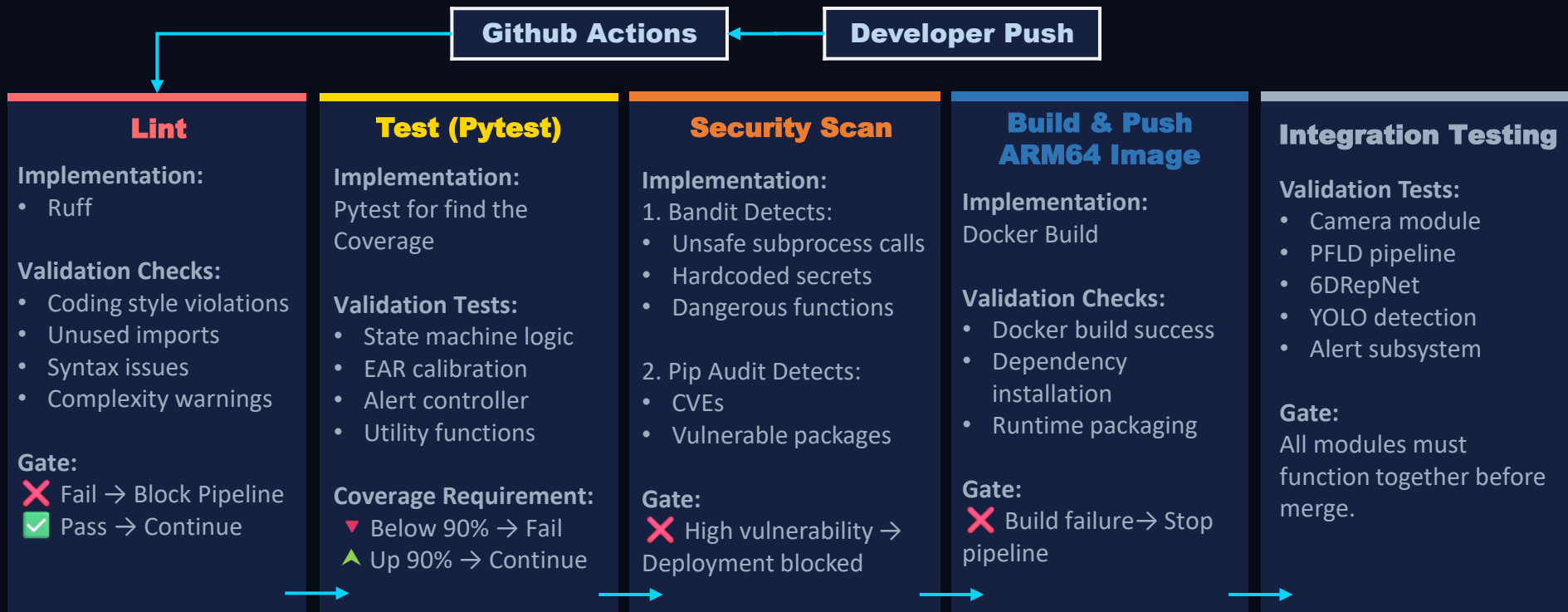
1. Developed an automatic calibration procedure during initialization.
2. Implemented a state machine: LED indicators; Buzzer alerts; Steering vibration

Result

- ✓ More robust drowsiness detection across different users
- ✓ End-to-end AI-to-hardware response system

CONTINUOUS INTEGRATION WORKFLOW

What happens when code is pushed? What quality gates are enforced? How do we prevent bad code?



Push Code → Lint → Coverage $\geq$ 90% → Security Scan → ARM64 Build → Integration Test → Merge Approved

CONTINUOUS DEPLOYMENT WORKFLOW

What happens when a release is deployed? How do we safely deploy good code?

Release Trigger

Implementation:

push:
tags:
- v*.*.*

Purpose:

- Deploy only official releases
- Prevent accidental production deployments
- Enable version-controlled deployment

Example:
v1.0.0

Self-Hosted Jetson Runner

Implementation:

runs-on:
[self-hosted, linux,
arm64, jetson]

Purpose:

- Execute deployment directly on Jetson Orin Nano
- Validate deployment on actual target hardware
- Avoid a rchitecture mismatch

Container Version Management

Implementation:

docker buildx imagetools
create

Purpose for Generate Deployment Tags:

v1.2.0
v1.2

Benefit:

- Easier version tracking
- Supports rollback
- No image rebuild required

Automated Deployment

Implementation:

deploy.sh

Purpose:

- Pull release image
- Replace previous container
- Apply production configuration
- Start new service

Benefit:

Fully automated deployment process.

Post-Deployment Health Check

Implementation:

curl
localhost:8000/healthz

Purpose:

- Application startup success
- Service availability
- Deployment integrity

Benefit:

Detect deployment failures automatically.

Release Tag → Jetson Runner → Version Control → Deploy Script → Health Check → Production

Production

TEST SET & METHODOLOGY

Three signals, one goal — detect danger before it becomes an accident

Test Set Preparation

Dataset Preparation:

- Recorded a dedicated validation video using a USB camera
- The validation video was not used during development or threshold tuning
- Ground-truth events were manually annotated

Ground Truth:

For each second of the video: Timestamp

→ Expected Driver State

Example:

00:15–00:20 Normal

00:21–00:25 Yawning

00:26–00:30 Drowsiness

Accuracy Measurement

Evaluation Procedure:

1. Run validation video through the complete DMS pipeline
2. Record system state predictions
3. Compare predictions against manually annotated ground truth
4. Compute classification metrics

Recall:

$TP / (TP + FN)$

Precision:

$TP / (TP + FP)$

F1 Score:

$2 \times \text{Precision} \times \text{Recall} / (\text{Precision} + \text{Recall})$

Performance Instrumentation

Latency Measurement:

Collected directly from the inference pipeline with logged:

p50 latency

p95 latency

p99 latency

FPS Measurement:

Processed Frames / Elapsed Time

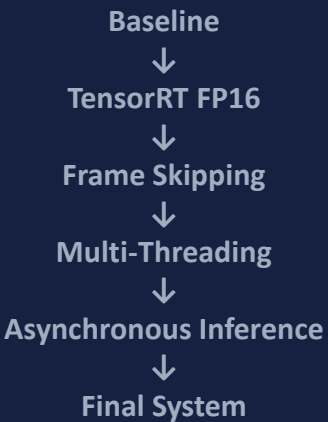
CPU / GPU / Power:

Using command: `tegrastat`

PERFORMANCE REQUIREMENTS & OPTIMIZATION JOURNEY

Which optimization mattered most?

Optimization System



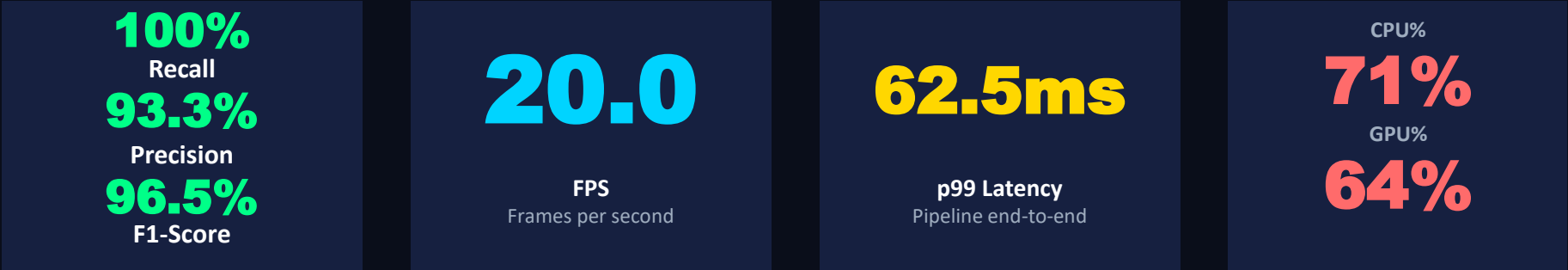
Optimization Step	Why	FPS	Latency (ms)	GPU Efficiency
TensorRT FP16	Faster inference	↑	↓	Better
Input Resolution Scaling	Lower computational cost	↑	↓	Better
Frame Skipping	Reduce redundant processing	↑	↓	Better
Multi-Threading	Parallel execution	↑	↓	Better
Async Inference	Remove blocking operations	↑	↓	Better

Key Contributor to Performance Improvement

- 1 Multi Threading
Improved CPU and GPU resource utilization
- 2 TensorRT FP16
Reduced model execution latency
- 3 Frame Skipping
Lowered computational workload

RESULTS — ACCURACY, LATENCY & RESOURCE USAGE

Final held-out test set — measured on Jetson Nano under sustained 5-minute load



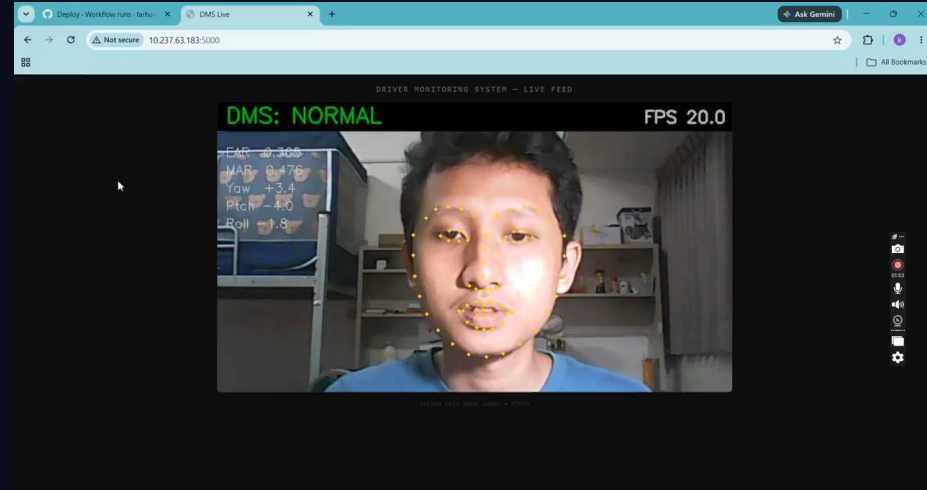
Metric	P50 (median)	p95	p99	Power (VDD_IN)
Pipeline Latency	48.9 ms	52.4 ms	62.5 ms	6.8W

Metric	Baseline	After
Latency	100.2 ms	62.5 ms
FPS	4 – 7 FPS	20 FPS
Power Consumption	7.6W	6.8W
CPU Usage	77%	71%
GPU Usage	98%	64%

Key Finding

- Achieved 20 FPS real-time performance
- Reduced p95 latency by approximately 47%
- GPU utilization decreased from 98% to 64% after optimization
- Power consumption remained below 7 W
- Final F1 Score reached 96.5%.

DEMO VIDEO



PROJECT RETROSPECTIVE

What We'd Do Differently

Perform EAR Calibration Research Earlier

What Happened : The original proposal used a fixed EAR threshold

Discover : Each person has different eye geometry

What's Changed : Design personalized calibration from the beginning

Why : This became one of the most important improvements

Expected Benefit : Reduced false detections and less rework later.

Automate Evaluation Pipeline

What Happened : Ground-truth labeling and validation involved significant manual effort

What Changed : Build an automated evaluation framework earlier

Why : Reduce manual verification workload

Expected Benefit : Faster iteration and reproducible testing.

Expand Driver Monitoring Scope

What Happened:

Current system focuses on:

- Drowsiness
- Distraction
- Yawning
- Phone usage

What Changed:

Include additional driver behaviors, like:

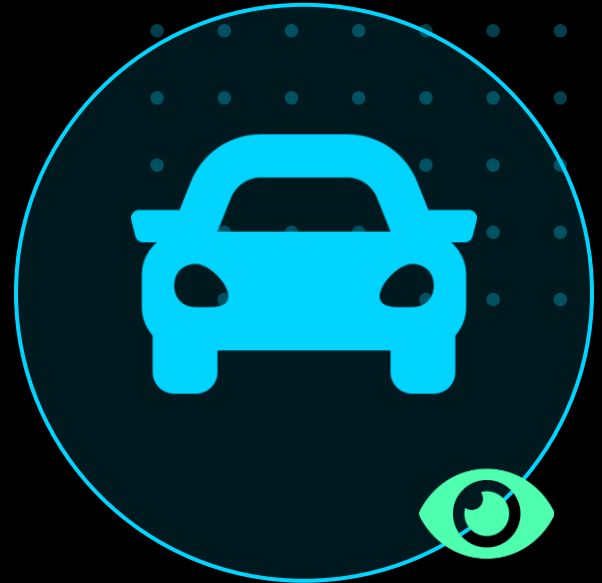
- Seatbelt detection
- Driver identity verification
- Smoking detection
- Emotion recognition

Expected Benefit:

More comprehensive driver safety monitoring.

Thank You

Question & Discussion



Special Thanks To:

- ❖ Prof. 宋建龍 (J.L. Sung) for his guidance, valuable feedback, and continuous support throughout the this course
- ❖ Mr. Dung Dinh Trong, our Teaching Assistant, for his technical assistance and helpful suggestions during the project development process
- ❖ Billy for generously providing the USB speaker used in our system testing and demonstrations
- ❖ Ourselves, for maintaining strong collaboration, overcoming technical challenges, and consistently supporting each other throughout the development of this project